BMMS Architecture

Board Minutes and Meeting Software (BMMS)

Field	Value
Owner	Gashpy Software
Consultant	Ganesh Shankar C S
Date	2026-10-18
Code baseline	branch `modernization`, tag `user-system-admin-3`
Database baseline	steps 1 to 18 installed (see `Database/README.md`)

This document describes the application as it is in the repository and the database as it is after steps 1 to 18. It replaces the earlier architecture and modernisation PDFs.

1. What BMMS does

BMMS runs the board and committee meeting cycle of a company:

- Masters: committees, members, committee membership (appointment and cessation dates), departments, designations, member types, subject types, decision types, users.
- Meetings: meeting master, invitees (meeting members), attendance and chairman.
- Agenda: items raised by users, approved first by their Manager and then by a Controller, finalized per meeting.
- Minutes: updated per agenda item, finalized and confirmed; action items are created at finalization and followed up.
- Alerts and mail: dashboard alerts, alert mail, notices, follow-up mail, free-form Send Mail.
- Reports: notices, minutes, follow-up and action taken, attendance, calendars, committee and meeting masters, sitting fees, audit trail.
- Utilities: reset password, backup database, reset database, SMTP settings, system settings.

2. Solution structure

`BMMS.sln`, three projects, all .NET Framework 4.8:

Project	Kind	Contents
`BMMS`	ASP.NET Web Forms application	47 `.aspx` pages under `Master/`, `Report/`, `DashBoard/`, `Error/` and the root; master pages; user controls; CSS and scripts; XML settings; `PageAccessModule.cs`
`BMMSBAL`	Class library (business layer)	one `*Mgr` class per area (Agenda, Meeting, Minutes, User, ...), mail (`Mail_Mgr`), PDF output (`PDF_Reports`, `PDFUtilities`, `HtmlToPdfBuilder`), Word output (`WordUtilities`), password hashing (`Common`)
`BMMSDAL`	Class library (data layer)	one `*DB` class per area; `CommonDB` runs the calls; `ConnectDB` reads the connection string

References run one way: `BMMS` → `BMMSBAL` → `BMMSDAL`. The pages never call the data layer directly.

Third-party libraries (tracked under `BMMS/bin` and `BMMSBAL/bin/Debug`, found there by the build): AjaxControlToolkit, FreeTextBox (rich-text editor), iTextSharp 4 (PDF), Microsoft Office Word interop (Word output).

3. Request flow

```
Browser
  │  HTTP (Web Forms postback)
  ▼
IIS / ASP.NET  ── PageAccessModule (every .aspx request)
  │                 1. public pages pass (login, logout, error)
  │                 2. a logged-in, ACTIVE user is required
  │                 3. the role must be listed for the page (XML/PageAccess.xml)
  ▼
Page code-behind (BMMS)
  │  Session["UserId"], Session["RoleCode"]
  ▼
*Mgr class (BMMSBAL)  ── mail, PDF, Word
  ▼
*DB class (BMMSDAL)  ── CommonDB: SqlCommand, stored procedure + parameters
  ▼
SQL Server database BMMS
```

4. Configuration

File	Purpose
`BMMS/Web.config`	ASP.NET settings; `connectionStrings` is read from `connectionStrings.config`
`BMMS/connectionStrings.config`	the connection string; not tracked (copy `connectionStrings.config.example`)
`BMMS/XML/AppSettings.xml`	page title, footer text, common messages, login-id and password lengths
`BMMS/XML/PageAccess.xml`	page-access policy: the roles allowed per page, and the mode (`LogOnly` or `Enforce`)
`BMMS/XML/PageSettings.xml`, `ReportSettings.xml`	page and report settings
SMTP settings	stored in the database, edited on the SMTP Server Config page
`App_Setting` table	system settings (step 16), edited on the System Settings page

5. Security

Login and passwords. Passwords are stored as PBKDF2 (RFC 2898) hashes, 100,000 iterations, a 16-byte random salt per password, in the form `iterations.salt.hash`. The login lookup and the password check are separate (step 1). A user logging in with the application's default password is sent straight to the first-login password change.

Account status. `Users.AccountStatus` is 1 (active) or 0 (inactive). Login refuses an inactive account after the password check, and `PageAccessModule` re-reads the status on every request, so a user deactivated while logged in is logged out on the next click.

Roles. `admin`, `controller`, `manager`, `user`, `mgmt`, `md_sect`. The menu and `PageAccess.xml` decide which pages a role opens; the stored procedures decide which changes a role may make.

Who may change which user (`dbo.fn_UserChangeRefusal`, step 13 onwards):

- Nobody can delete, deactivate or change the role of their own account.
- The last active ordinary Admin and the last active Controller cannot be deleted, deactivated or given another role (checked under a lock, inside the same transaction).
- Admin: any user. Controller: any user except Admin and Controller users. Other roles: their own details and password only.
- The acting user is the logged-in user, passed as `@ActingUserId`; a missing or inactive acting user is refused.

The protected system account (step 18). One Admin user, `Users.IsSystem = 1` (the "admin" account), cannot be edited, deleted, deactivated or given another role, even by itself; only it can change its own password. It takes no part in the workflow (no alerts, no action items, no approvals). A lost password is recovered by the DBA as described in `Database/README.md`.

Result contract. Every protected procedure returns one row `Result, Reason, ReasonDetail` (Result 1 done, 0 refused). The page turns the Reason into a plain sentence. A database error is never reported as a refusal: it is re-raised, reaches the error page and is written to `BMMS\Log\yyyy_MM_dd_ErrorLog.txt`.

6. Workflows

Agenda approval.

```
User adds item ──► AC alert to the user's Manager
Manager approves ──► AMA alert to the Controllers      (or AMR: rejected)
Controller approves ──► ACA, item ready for the meeting (or ACR: rejected)
Controller finalizes the agenda (AgendaStatus Open ──► Close)
```

A user whose manager is inactive cannot add an item (Reason 11). Once the agenda is finalized, items cannot be added, edited, deleted, approved or rejected (Reason 5), except by an active Controller while the minutes are open and the System Settings option allows it.

Meeting and minutes.

```
Meeting created ──► members tagged ──► attendance and chairman (only on or after the meeting date)
Minutes updated ──► minutes finalized (MinutesStatus Open ──► F, action items created)
                ──► minutes confirmed (F ──► C)
```

- After attendance is recorded, the meeting date and committee are locked; after agenda items exist, the committee is locked.
- After the minutes are finalized, meeting members, attendance, chairman and agenda items are locked.
- After the minutes are confirmed, the minutes text is locked.
- Each finalize or confirm step runs only from the state before it, so a second click or a second tab changes nothing.
- Committee members count for a meeting only between their appointment and cessation dates.

Alerts and action items. New alerts, action items and alert mail go to active users only, never to the system account. An action item whose agenda creator is inactive goes to the active Controllers.

7. Database

SQL Server database `BMMS`. Almost every call from the data layer is a stored procedure (`spr_*`). The exceptions are some report queries built in `ReportsMgr` (run through `CommonDB.GetDataTableFromQuery`) and the plain T-SQL database backup (`Sql_Backup_Mgr`).

Main tables. `Users`, `Roles`, `Department`, `Designation`; `Committee`, `Committee_Member`, `Member`, `Member_Type`, `State_Master`; `Meeting`, `Meeting_Member`, `Attendance`; `Agenda`, `Minutes`, `Action_Item`, `ActionItem_User_Map`; `Alert_Master`, `Alert_User_Map`, `Alert_Type`, `Source_Type`; `Subject_Type`, `Decision_Type`; `App_Setting`; audit tables `Audit_Trail_Master`, `Audit_Trail_SendMail`, `Audit_Trail_SqlBackup`, `Audit_Trial_Login`.

Integrity. Step 4 added primary keys to the two tables that had none (`Source_Type`, `Audit_Trail_SendMail`). Foreign keys: committee keys (step 5), minutes, action-item, chairman and member-state keys (step 7), `FK_Users_ManagerId` (step 9) and 21 keys from the user-id columns to `Users` (step 11). `Users.AccountStatus` NOT NULL, default 1, CHECK 0 or 1; `Users.IsSystem` BIT NOT NULL, default 0, CHECK.

Change history: steps 1 to 18. Each step is one script in `Database/`, installed in order, with its own tests; all are installed on `BMMS`.

Step	Script	Change
1	BMMS-Fix-Login-Procedures	login lookup split for hashed passwords
2	BMMS-Fix-Delete-Procedures	delete procedures check references
3	BMMS-Fix-ResetDatabase-Seeds	Reset Database seeds hashed passwords, no-manager as NULL
4	BMMS-Add-PrimaryKeys	missing primary keys
5	BMMS-Add-ForeignKeys-Safe	committee keys
6	BMMS-Fix-FK-Procedures	7 procedures made key-safe
7	BMMS-Add-ForeignKeys-Step3	4 keys
8	BMMS-Fix-MinutesFinalize-Procedure	no duplicate action items
9	BMMS-Add-ForeignKeys-Step4	manager key
10	BMMS-Fix-UserDelete-Procedure	a user with history cannot be deleted
11	BMMS-Add-ForeignKeys-Step5	21 user-id keys
12	BMMS-Fix-Attendance-FutureMeeting	no attendance before the meeting date
13	BMMS-Fix-UserProtection-Stage1	who may change which user
14	BMMS-Fix-UserStatus-Stage2a	active / inactive users
15	BMMS-Fix-MeetingLocks	meeting-state locks, part 1
16	BMMS-Fix-MeetingLocks-Step16	meeting locks, part 2; System Settings
17	BMMS-Fix-UserStatus-Stage2b	inactive users get no new work
18	BMMS-Fix-SystemAccount-Stage3	protected system account

`BMMS-Check-InstalledSteps.sql` (read only) shows which steps a database has. The run order, the re-run rules and the release notes for each step are in `Database/README.md`.

8. Build, test and release

- **Build:** Visual Studio, Rebuild Solution (`BMMS.sln`), 0 errors.

- **Database tests:** every step has a test script that runs inside a transaction and rolls back; they run on a test copy (`BMMS_Test` or `BMMS_bak1`), never first on `BMMS`.
- **Release:** back up `BMMS`, run the step's script, then deploy the matching build straight away (the scripts say when the order matters).
- **Source control:** branch `modernization`; one tag per release; files are staged by explicit path; rebuilt binaries stay out of commits.

9. Known limitations

- `Minutes.CreatedBy` / `UpdatedBy` have no key: two procedures write a hard-coded user id there.
- Old mail and login audit rows can show the wrong user name (ids of users deleted long ago are reused).
- A user whose manager is a Controller, or who has no manager, cannot get an agenda item through the Manager approval.
- Some report and certificate texts are still fixed in the code; making them company-specific is planned.